SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

ASSIGNMENT 2 – CO2

Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Student Name	S.Dharshan Vishal	**Register Number**	192321174
Department / Section	Information Technology	**Academic Year / Semester**	2026-2027
Faculty Name	Dr.P.Murali	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Grey Level Transformation Analysis	4	3.5	Three transformations on 10 pages; contrast 39.5 → 69.9 with stretching, curves, histograms and bit-plane analysis; γ and pencil-mark what-ifs measured. Pretrained enhancement model reasoned, not run.
2	Histogram Processing and Enhancement	4	3.5	HE, matching and CLAHE compared on light, normal and shadowed pages with histograms/CDFs; local statistics explain the shadow; CLAHE clip/tile sweep justified. Deep model comparison not run.
3	Spatial Filtering and Image Smoothing	5	4.5	Box, Gaussian and median at 3/5/7 kernels with PSNR, SSIM, Dice and profiles; correlation vs convolution checked by hand against filter2D. DnCNN / diffusion reasoned only.
4	Image Sharpening and Edge Enhancement	5	4.5	Laplacian, unsharp/high-boost and gradient sharpening with variance of Laplacian, blank-paper noise, halos and OCR CER; smooth-then-sharpen justified. Deep deblurring reasoned only.
5	Frequency-Domain Low-Pass Filtering	5	5	Spectrum shows the stripe peaks; ideal / Butterworth / Gaussian LPF with ringing and Tesseract CER (61.6 % → 1.9 %); Gaussian equivalence and padding wrap-around measured. Excellent.
6	Frequency-Domain High-Pass Filtering	4	3.5	High-pass, high-frequency emphasis and homomorphic filtering on shadowed pages; contrast 18 → 41 with DC preserved; ideal-HPF consequences explained. Learned shadow removal not run.
7	Point, Line, and Edge Detection	5	4.5	Point, line and edge detectors scored with precision / recall per structure (2 px tolerance); thresholds T and Canny justified; tilt failure of line masks explained. HED not run.

8	Image Thresholding and Binarization	4	3.5	Otsu, global and adaptive thresholds on 10 pages with Dice (0.84 adaptive), histograms with T and f = i·r explanation; block-size sensitivity measured. DIBCO model reasoned only.
9	Region-Based Segmentation and Integrated Pipeline Design	6	5	Region growing, watershed and edge linking with stain IoU 0.89 and corner error 2.1 px; integrated pipeline 1.46 s with stage times, block diagram and decision table. SAM 2 not run.
10	Smart Scanner Adaptation for High-Resolution Phone Photographs	8	7.5	Impact analysis and audit lead to MODIFY: Dice 0.79, blue notes kept, 119 ms per page. Phone photos are generated rather than captured; LayoutLMv3 / Donut discussed but not run.
	TOTAL	**50**	**45**	**Percentage: 90 %**

OVERALL REMARKS

Very good work: every task applies the three techniques on the same pages, measures them with suitable metrics (contrast, PSNR/SSIM, Dice, IoU, Tesseract CER, time) and ends with a justified decision. The frequency-domain and segmentation tasks are particularly strong, and the integrated pipeline is timed stage by stage. Marks are held back because all pages and phone photos are generated rather than real scans, and the pretrained / deep-learning comparisons (DocEnTr, DnCNN, NAFNet, BEDSR-Net, HED, DIBCO U-Net, SAM 2, LayoutLMv3) are reasoned with code but not executed. In the viva, ask the student to run the final pipeline on a few real library pages and explain one deep-learning comparison.

M.Samanvi		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Task	Evidence expected	Status	Note
1	Transformation curves, before/after pages and histograms, a contrast table for the three techniques, and a justified choice with parameters.	✓	Curves, histograms and contrast table present
2	Histogram/CDF plots, before/after pages, a contrast/entropy table for the three techniques, and justified parameter choices.	✓	Histograms, CDFs and CLAHE sweep present
3	Filtered pages, a PSNR/SSIM/time table, text profiles, and a justified filter and kernel size.	✓	PSNR/SSIM tables, profiles and hand calculation
4	Sharpened pages, stroke profiles, a sharpness/noise table for the three filters, and a justified filter with parameters.	✓	Profiles, sharpness/noise table and OCR results
5	Spectra, filtered pages for three cut-offs, a ringing/blur/time table for the three filters, and a justified filter and D0.	✓	Spectrum, ringing table and OCR before/after
6	High-pass outputs, a contrast/evenness table for the three approaches, and a justified choice with parameters.	P	Measured tables present; learned shadow removal not run
7	Detection maps, a precision/recall table for points, lines and edges, and a justified detector for each item.	✓	Detection images and precision/recall per structure

8	Binary pages, annotated histograms, a Dice table for the three techniques, and a justified thresholding method.	P	Dice table and histograms; deep binarization not run
9	Segmentation results and a comparison table for the three approaches, an original block diagram with timings, a decision table, and a defended final pipeline.	✓	IoU/corner table, block diagram and decision table
10	Three adaptation strategies applied and compared with measured evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.	P	Strategies measured on generated phone photos

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Evidence item	Status	Observation
Source code (Python/OpenCV, commented)	✓	Commented code for every technique; page generator shared by all tasks
Test inputs (pages, phone photos, ground-truth masks)	P	Generated pages with exact ground truth; no real scanned pages
Expected behaviour (predicted contrast, ringing, Dice)	✓	Predictions stated and checked against measurements (Task 9)
Actual output (measured contrast, PSNR, Dice, CER, time)	✓	Measured tables in every task, incl. Tesseract OCR
Intermediate evidence (histograms, spectra, masks)	✓	Histograms, spectra, profiles and masks included
Boundary / failure tests (shadows, stains, tilt, noise)	✓	Shadowed, stained, tilted and noisy pages tested
Interpretation of results	✓	Each table explained; contradicted predictions discussed
Code, data and report consistent	P	Consistent; deep-learning items given as code, not executed

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 10 tasks answered	☒ Yes ☐ Partly ☐ No
Three techniques applied and compared in every task	☒ Yes ☐ Partly ☐ No
Commented Python/OpenCV source code for every experiment	☒ Yes ☐ Partly ☐ No
Results obtained from own code execution or measurements (deep-learning items not executed)	☐ Yes ☒ Partly ☐ No
Same data and conditions used for all three techniques in a task	☒ Yes ☐ Partly ☐ No
Calculations and measurements shown with units	☒ Yes ☐ Partly ☐ No
Tables and graphs for experimental results	☒ Yes ☐ Partly ☐ No
Figures / screen outputs included in the submission	☒ Yes ☐ Partly ☐ No
Justified final decision with limits stated in every task	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended (run the pipeline on real library pages; explain one deep-learning comparison)	☒ Yes ☐ No

OMPUTER VISION

ASSIGNMENT 2 – CO2

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Assignment Title: Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library (Total Marks – 50)

Scenario – Smart Document Scanner for the College Library

The college library wants to digitize its old printed question papers and lecture notes. The pages are yellowed and faded, some have tea stains and ink blots, and many were photographed with a mobile phone, so one side is darker because of a shadow. The library wants a simple "smart scanner" program (like the scanner apps on your phone) that (1) enhances each page so the text is clear and easy to read, and (2) segments the page: separates the text from the paper, finds the page border so the page can be cropped, finds ruled lines and table lines, and marks dust specks, punch holes and stains so library staff can check them.

You are the vision engineer. In every task, three candidate techniques are given. You must apply all three, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

1. INTRODUCTION

Traffic signs play an important role in maintaining road safety and controlling traffic movement. They provide information about traffic rules, speed limits, warnings, restrictions, and pedestrian crossings. Drivers depend on these signs to make correct decisions while driving.

With the development of smart cities and intelligent transportation systems, computer vision technology can be used to automatically detect and analyze traffic signs. Cameras installed on roads and vehicles capture images of traffic signs. These images are processed using image processing and machine learning techniques to identify the signs and understand their condition.

The proposed Smart Traffic Sign Detection System aims to detect different types of traffic signs, such as:

1. STOP signs.
2. Speed-limit signs.
3. No Entry signs.
4. Pedestrian crossing signs.

However, detecting traffic signs in real-world environments is challenging because images are captured under different environmental conditions.

1.1 Problems in Traffic Sign Detection

1. Heavy sunlight

During daytime, strong sunlight can create excessive brightness and reflections on traffic signs. This may hide important details and reduce image quality.

2. Night-time conditions

At night, images may become dark because of insufficient lighting. Traffic signs may not be clearly visible, making detection difficult.

3. Rainy weather

Rain can create water droplets, reflections, and noise in captured images. These effects reduce the clarity of traffic signs.

4. Faded traffic signs

Some traffic signs lose their original colour because of weather conditions and long-term exposure to sunlight. Faded signs become difficult to identify using simple colour-based methods.

5. Partial obstruction

Traffic signs may be partially covered by tree branches, electric poles, vehicles, or other objects. This makes it difficult to identify the complete sign.

6. Motion blur

When a vehicle moves at high speed, the camera may capture blurred images. This can reduce the accuracy of boundary detection and recognition.

1.2 Objectives of the System

The main objectives of the Smart Traffic Sign Detection System are:

- To improve the visibility of traffic signs using image enhancement techniques.
- To separate traffic signs from the surrounding background.
- To detect the boundaries and important regions of traffic signs.
- To identify faded, damaged, and partially hidden traffic signs.
- To prepare clear and properly processed images for further traffic sign recognition.
- To improve the reliability of traffic sign detection under different environmental conditions.

1.3 Applications

The proposed system can be used in:

- Smart city traffic monitoring.

- Advanced Driver Assistance Systems (ADAS).
- Autonomous vehicles.
- Road safety monitoring.
- Automatic traffic sign inspection.
- Intelligent transportation systems.
- Road maintenance and infrastructure management.

2. SYSTEM ASSUMPTIONS AND REQUIREMENTS

2.1 Assumptions

The following assumptions are considered while developing the proposed system:

1. Cameras are installed on roads or vehicles to capture traffic sign images.
2. The captured images are available in RGB or BGR colour format.
3. The system processes STOP, speed-limit, No Entry, and pedestrian crossing signs.
4. Images may contain noise, shadows, blur, reflections, and partial obstruction.
5. Python and OpenCV are used for implementing image processing techniques.
6. A labelled dataset is available for training and evaluating machine learning models.
7. All candidate techniques are tested using the same input images for fair comparison.
8. The system provides processed images and candidate sign regions for a later recognition stage.

2.2 Functional Requirements

Requirement	**Description**
Image enhancement	Improve image visibility and contrast.
Image segmentation	Separate traffic signs from the background.
Boundary detection	Identify the outer boundaries of signs.
Condition detection	Identify faded, damaged, and occluded signs.
Output preparation	Generate clear images for recognition.

2.3 Non-Functional Requirements

- Accuracy: The system should detect traffic signs correctly.
- Speed: Processing should be suitable for the intended camera frame rate.
- Reliability: The system should work under different lighting and weather conditions.
- Scalability: It should support multiple cameras.
- Maintainability: Individual processing techniques should be replaceable.
- Robustness: It should handle noise, blur, and partial obstruction.

2.4 Evaluation Criteria

The techniques are compared using the following criteria:

Metric	Description
Precision	Measures how many detected regions are correct.
Recall	Measures how many actual traffic signs are detected.
F1-score	Balances precision and recall.
IoU	Measures overlap between predicted and actual regions.
Processing time	Measures the time required to process an image.
Robustness	Measures performance under different environmental conditions.

3. TASK 1 – IMAGE ENHANCEMENT

3.1 Introduction

Image enhancement is an important preprocessing step in computer vision. It improves the visual quality of captured images and makes important objects easier to identify.

In the Smart Traffic Sign Detection System, images may be affected by heavy sunlight, darkness, rain, shadows, and low contrast. These conditions can make traffic signs difficult to detect.

Image enhancement techniques improve brightness, contrast, and visibility. The enhanced images are then passed to the segmentation stage.

The three candidate techniques considered for this task are:

1. Gamma Correction.
2. Histogram Equalization.
3. Contrast Limited Adaptive Histogram Equalization (CLAHE).

3.2 Technique 1: Gamma Correction

Gamma correction is an image enhancement technique used to adjust the brightness of an image.

It modifies pixel intensity values using a mathematical power-law transformation.

The formula is:

$$I_{out} = 255 \left(\frac{I_{in}}{255} \right)^{\gamma}$$

Where:

- I_{in} = Original pixel intensity.
- I_{out} = Enhanced pixel intensity.
- γ = Gamma value.

Advantages

- Simple to implement.
- Requires less processing time.
- Useful for correcting underexposed images.
- Suitable for real-time applications.

Disadvantages

- Uses a global intensity transformation.
- May not handle images containing both bright and dark regions.
- Can increase noise in dark areas.

- Requires proper gamma selection.

Python Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the input image
img = cv2.imread("traffic_scene.jpg")

# Gamma correction function
def gamma_correction(image, gamma=1.25):

    table = np.array([
        ((i / 255.0) ** (1.0 / gamma)) * 255
        for i in range(256)
    ]).astype("uint8")

    return cv2.LUT(image, table)

# Apply gamma correction
result = gamma_correction(img, 1.25)

# Display the output
plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.subplot(1, 2, 2)
plt.title("Gamma Corrected Image")
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.show()
```

3.3 Technique 2: Histogram Equalization

Histogram equalization is a contrast enhancement technique that redistributes image intensity values.

It improves the overall contrast by spreading intensity values across the available range.

It is particularly useful when an image has low global contrast.

Working Principle

1. Read the input image.
2. Convert the image into grayscale or a suitable luminance representation.
3. Calculate the image histogram.
4. Redistribute the intensity values.
5. Generate the enhanced image.

Advantages

- Simple and fast.
- Improves overall image contrast.
- Does not require complex model training.
- Useful for low-contrast images.

Disadvantages

- May increase noise.
- Can over-enhance bright regions.
- Does not adapt separately to local regions.
- May reduce natural image appearance.

Python Code

```
import cv2

import matplotlib.pyplot as plt

# Read image

img = cv2.imread("traffic_scene.jpg")

# Convert to YCrCb colour space

ycrcb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)

# Split channels

y, cr, cb = cv2.split(ycrcb)

# Apply histogram equalization

equalized = cv2.equalizeHist(y)

# Merge channels

merged = cv2.merge([equalized, cr, cb])

# Convert back to BGR

result = cv2.cvtColor(merged, cv2.COLOR_YCrCb2BGR)
```

```
# Display output
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Histogram Equalization")
plt.axis("off")
plt.show()
```

3.4 Technique 3: CLAHE

Contrast Limited Adaptive Histogram Equalization (CLAHE) is an advanced version of histogram equalization.

Unlike global histogram equalization, CLAHE divides an image into smaller regions called tiles. It enhances contrast separately in each region.

It also uses a contrast limit to reduce excessive enhancement and noise amplification.

Working Principle

1. Read the input image.
2. Convert it into a luminance-based colour representation.
3. Divide the image into small regions.
4. Apply adaptive histogram equalization to each region.
5. Limit excessive contrast amplification.
6. Combine the enhanced regions.
7. Generate the final image.

Advantages

- Improves local contrast.
- Useful for uneven illumination.
- Helps reveal details in shadowed regions.
- Suitable for images captured under different lighting conditions.

Disadvantages

- Requires parameter tuning.
- May increase noise if the contrast limit is too high.
- Requires more computation than basic histogram equalization.

Python Code

```
import cv2
import matplotlib.pyplot as plt
# Read image
img = cv2.imread("traffic_scene.jpg")
# Convert to YCrCb
ycrcb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
# Split channels
y, cr, cb = cv2.split(ycrcb)
# Create CLAHE object
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
# Apply CLAHE
enhanced_y = clahe.apply(y)
# Merge channels
result = cv2.merge([enhanced_y, cr, cb])
# Convert back to BGR
result = cv2.cvtColor(result, cv2.COLOR_YCrCb2BGR)
# Display result
```

```
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))

plt.title("CLAHE Enhanced Image")

plt.axis("off")

plt.show()
```





3.5 Comparison of Three Techniques

Parameter	**Gamma Correction**	**Histogram Equalization**	**CLAHE**
Processing speed	High	High	Moderate
Contrast improvement	Global brightness adjustment	Global contrast	Local contrast
Shadow handling	Limited	Moderate	Good
Noise amplification	Possible	Possible	Possible
Parameter tuning	Required	Minimal	Required
Uneven illumination	Limited	Limited	Better suited

3.6 Selection of the Best Technique

CLAHE is selected as the most suitable primary technique for the proposed Smart Traffic Sign Detection System.

The main reason is that traffic sign images may contain both bright and dark regions. Global techniques may improve the overall brightness but may not preserve details in all regions. CLAHE enhances local contrast and helps reveal important traffic sign details under shadows and uneven lighting.

Gamma correction can also be used when the complete image is too dark. However, CLAHE provides a better starting point for images with mixed illumination.

3.7 Evidence and Performance Evaluation

To justify the selection, all three techniques should be tested using the same collection of traffic sign images.

The evaluation should include:

- Images captured during heavy sunlight.
- Images captured at night.
- Images captured during rain.
- Images affected by shadows.
- Images containing faded traffic signs.

The performance can be compared using local contrast, image quality, percentage of saturated pixels, and the accuracy of the segmentation stage that follows.

Actual numerical results should be recorded after running the experiments. CLAHE is the recommended technique based on its local contrast enhancement capability, but its performance must be verified using the target dataset.

3.8 Task 1 Conclusion

Image enhancement is an important preprocessing step in traffic sign detection. Gamma correction, histogram equalization, and CLAHE are three useful techniques for improving image visibility. Gamma correction is suitable for adjusting overall brightness, while histogram equalization improves global contrast. CLAHE improves local contrast and is more

suitable for images containing uneven illumination and shadows. Therefore, CLAHE is selected as the primary technique for the proposed system. It can improve the visibility of traffic signs and provide better input for the next stage of image segmentation.

TASK 2 – TRAFFIC SIGN SEGMENTATION

2.1 Introduction

Image segmentation is an important process in computer vision. It is used to divide an image into different regions based on colour, intensity, texture, or other visual characteristics.

In the Smart Traffic Sign Detection System, segmentation is used to separate traffic signs from the surrounding background. The captured road images may contain buildings, trees, vehicles, pedestrians, road markings, and other objects. These background objects can make traffic sign detection difficult.

The main purpose of segmentation is to identify the pixels that belong to traffic signs and separate them from unnecessary background information.

For example, when a camera captures a red STOP sign, the segmentation process identifies the red region and creates a separate mask. This mask helps the system locate the traffic sign more easily.

Segmentation is useful for identifying different types of traffic signs, including:

1. STOP signs.
2. Speed-limit signs.
3. No Entry signs.
4. Pedestrian crossing signs.

In this task, three segmentation techniques are considered and compared:

1. Fixed Colour Thresholding.
2. HSV Colour Segmentation.

3. GrabCut Segmentation.

2.2 Objectives of Image Segmentation

The main objectives of image segmentation are:

- To separate traffic signs from the background.
- To identify important colour regions.
- To remove unnecessary background information.
- To generate binary masks for traffic sign regions.
- To improve the accuracy of boundary detection.
- To prepare segmented images for further processing.
- To reduce the complexity of the recognition process.

2.3 Technique 1: Fixed Colour Thresholding

Introduction

Fixed Colour Thresholding is a basic image segmentation technique used to separate objects based on predefined colour values.

In this method, specific lower and upper colour limits are selected. Pixels that fall within the selected range are considered foreground pixels, while other pixels are considered background pixels.

For example, red and blue colour ranges can be used to identify traffic signs.

The technique is simple and requires less computational power.

Working Principle

The working process of fixed colour thresholding involves the following steps:

1. Read the input traffic sign image.
2. Identify the required colour range.
3. Set the lower and upper threshold values.
4. Compare every pixel with the selected colour range.

5. Generate a binary mask.
6. Separate the selected colour regions from the background.
7. Display the segmented traffic sign.

Mathematical Representation

A binary threshold mask can be represented as:

$$M(x,y) = \begin{cases} 255, & L \leq I(x,y) \leq U \\ 0, & \text{otherwise} \end{cases}$$

Where:

- $M(x,y)$ = Output mask.
- $I(x,y)$ = Input pixel value.
- L = Lower threshold.
- U = Upper threshold.
- 255 represents the foreground.
- 0 represents the background.

Advantages

1. Easy to understand and implement.
2. Requires less processing time.
3. Suitable for simple images.
4. Does not require model training.
5. Useful for identifying clearly coloured traffic signs.

Disadvantages

1. Sensitive to lighting variations.
2. May fail when traffic signs are faded.

3. Shadows can change the detected colour values.
4. Similar background colours may create false detections.
5. Not suitable as the only segmentation method in complex road environments.

Applications

- Identifying red STOP signs.
- Detecting blue traffic signs.
- Separating objects based on predefined colour ranges.
- Basic traffic sign segmentation.

2.4 Technique 2: HSV Colour Segmentation

Introduction

HSV stands for Hue, Saturation, and Value. It is a colour representation technique that separates colour information from brightness.

Unlike RGB colour representation, HSV makes it easier to select specific colour regions.

The three components are:

1. Hue (H)

Hue represents the actual colour, such as red, blue, green, or yellow.

2. Saturation (S)

Saturation represents the intensity or purity of a colour. Highly saturated colours appear more vivid, while low-saturation colours appear faded or grey.

3. Value (V)

Value represents the brightness of the image. Higher values indicate brighter pixels, while lower values indicate darker pixels.

Working Principle

The HSV segmentation process involves:

1. Read the input traffic sign image.

2. Convert the image from BGR or RGB into HSV colour space.
3. Select the required hue, saturation, and value ranges.
4. Apply colour thresholding.
5. Generate a binary segmentation mask.
6. Apply morphological operations to remove small noise.
7. Extract the traffic sign region from the original image.

Mathematical Representation

$$M(x,y) = \begin{cases} 255, & H_L \le H \le H_U \\ & S_L \le S \le S_U \\ & V_L \le V \le V_U \\ 0, & \text{otherwise} \end{cases}$$

Where:

- H represents Hue.
- S represents Saturation.
- V represents Value.
- L represents the lower limit.
- U represents the upper limit.

Advantages

1. Separates colour information from brightness.
2. Useful for detecting red and blue traffic signs.
3. Easy to implement using OpenCV.
4. Provides better control over colour selection than simple RGB thresholding.
5. Suitable for generating traffic sign candidate regions.

Disadvantages

1. Faded signs may have low saturation.

2. Shadows and reflections can affect segmentation.
3. Similar colours in the background may produce false detections.
4. Threshold values require proper tuning.
5. It may not identify signs that have lost their original colours.

Applications

- Red traffic sign detection.
- Blue traffic sign detection.
- Colour-based object segmentation.
- Traffic sign candidate extraction.

2.5 Technique 3: GrabCut Segmentation

Introduction

GrabCut is an advanced image segmentation technique used to separate foreground objects from the background.

It uses colour information and statistical models to distinguish foreground and background regions.

Unlike simple colour thresholding, GrabCut considers the relationship between neighbouring pixels and the overall appearance of the object.

It generally starts with a rectangle around the object. The algorithm then repeatedly estimates foreground and background regions and improves the segmentation.

Working Principle

The GrabCut segmentation process involves:

1. Read the input image.
2. Select a rectangular region around the traffic sign.
3. Initialize foreground and background models.
4. Analyse the colour information.

5. Estimate foreground and background pixels.
6. Refine the segmentation through repeated iterations.
7. Generate the final segmented image.

Main Components of GrabCut

1. Foreground Model

It represents the colour characteristics of the object that needs to be extracted.

2. Background Model

It represents the colour characteristics of the surrounding background.

3. Iterative Optimization

The algorithm repeatedly improves the separation between foreground and background.

4. Final Mask

The output is a mask that identifies foreground and background pixels.

Advantages

1. Can handle complex backgrounds better than simple fixed thresholds.
2. Uses foreground and background colour information.
3. Can produce detailed segmentation results.
4. Useful when the object has irregular boundaries.
5. Can refine an initial candidate region.

Disadvantages

1. Requires an initial rectangle or mask.
2. More computationally expensive than simple thresholding.
3. Performance depends on the quality of initialization.
4. May fail when foreground and background colours are very similar.
5. Not always suitable for processing every full video frame in real time.

Applications

- Traffic sign extraction.
- Foreground and background separation.
- Object segmentation.
- Image editing and object isolation.

2.6 Comparison of Three Segmentation Techniques

Parameter	**Fixed Colour Thresholding**	**HSV Segmentation**	**GrabCut**
Working principle	Predefined colour limits	HSV colour ranges	Foreground/background modelling
Processing speed	High	High	Moderate
Complexity	Low	Low to moderate	High
Lighting sensitivity	High	Moderate to high	Depends on initialization and image
Background separation	Basic	Good for distinctive colours	Can be more detailed
Parameter tuning	Required	Required	Initial rectangle and parameters
Real-time suitability	Good	Good	Depends on hardware and ROI size

2.7 Selection of the Best Technique

Among the three techniques, HSV Colour Segmentation is selected as the primary segmentation method for the proposed Smart Traffic Sign Detection System.

The main reason for selecting HSV is that many traffic signs use distinctive colours, such as red and blue. HSV separates colour information from brightness, making it easier to identify these colour regions.

Fixed Colour Thresholding is simple and fast, but its performance may decrease when lighting conditions change.

GrabCut can provide more detailed foreground extraction, but it requires an initial rectangle and additional processing time.

HSV segmentation provides a practical balance between speed, simplicity, and colour-based detection.

For better results, HSV segmentation can be combined with GrabCut. HSV first identifies candidate traffic sign regions, and GrabCut can refine selected regions when necessary.

2.8 Evidence and Performance Evaluation

To justify the selection of HSV segmentation, all three techniques should be tested using the same traffic sign images.

The test images should include:

- STOP signs captured during sunlight.
- Speed-limit signs captured during night-time.
- No Entry signs affected by shadows.
- Pedestrian crossing signs captured during rain.
- Faded and partially covered traffic signs.

The following evaluation measures can be used:

1. Intersection over Union (IoU)

IoU measures the overlap between the predicted segmentation and the actual annotated traffic sign region.

$$IoU = \frac{Area\ of\ Intersection}{Area\ of\ Union}$$

A higher IoU indicates better segmentation overlap.

2. Precision

Precision measures how many pixels identified as foreground actually belong to the traffic sign.

$$Precision = \frac{TP}{TP + FP}$$

3. Recall

Recall measures how many actual traffic sign pixels are correctly identified.

$$Recall = \frac{TP}{TP + FN}$$

4. Processing Time

Processing time measures how long the technique takes to generate the segmentation mask.

The final selection should consider segmentation quality and processing time together.

Numerical results should be added after running the techniques on the labelled dataset.

2.9 Practical Applications in the Proposed System

Image segmentation supports the following operations in the proposed system:

1. Separating traffic signs from road backgrounds.
2. Identifying red and blue sign regions.
3. Removing unnecessary background pixels.
4. Preparing candidate traffic sign regions.

5. Supporting contour and boundary detection.
6. Improving the input quality for further recognition.

2.10 Task 2 Conclusion

Image segmentation is an important stage in the Smart Traffic Sign Detection System because it separates traffic signs from complex road backgrounds. Fixed Colour Thresholding, HSV Colour Segmentation, and GrabCut are three useful techniques for this purpose. Fixed Colour Thresholding is simple and fast but is sensitive to lighting changes. HSV Segmentation provides better control over colour identification and is useful for detecting red and blue traffic signs. GrabCut provides more detailed foreground and background separation but requires proper initialization and additional processing. Therefore, HSV Colour Segmentation is selected as the primary technique because it provides a practical balance between speed, simplicity, and performance. It can generate traffic sign candidate regions that are useful for the next stage of boundary detection and analysis.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("traffic_sign.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# 1. Fixed Colour Thresholding
red = cv2.inRange(img, (100, 0, 0), (255, 100, 100))
```

```
# 2. HSV Segmentation
hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)

mask1 = cv2.inRange(hsv, (0, 70, 50), (10, 255, 255))
mask2 = cv2.inRange(hsv, (170, 70, 50), (180, 255, 255))
mask = cv2.bitwise_or(mask1, mask2)

# 3. GrabCut Segmentation
bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
gc_mask = np.zeros(img.shape[:2], np.uint8)

rect = (10, 10, img.shape[1]-20, img.shape[0]-20)
bgd = np.zeros((1, 65), np.float64)
fgd = np.zeros((1, 65), np.float64)

cv2.grabCut(bgr, gc_mask, rect, bgd, fgd, 5,
            cv2.GC_INIT_WITH_RECT)

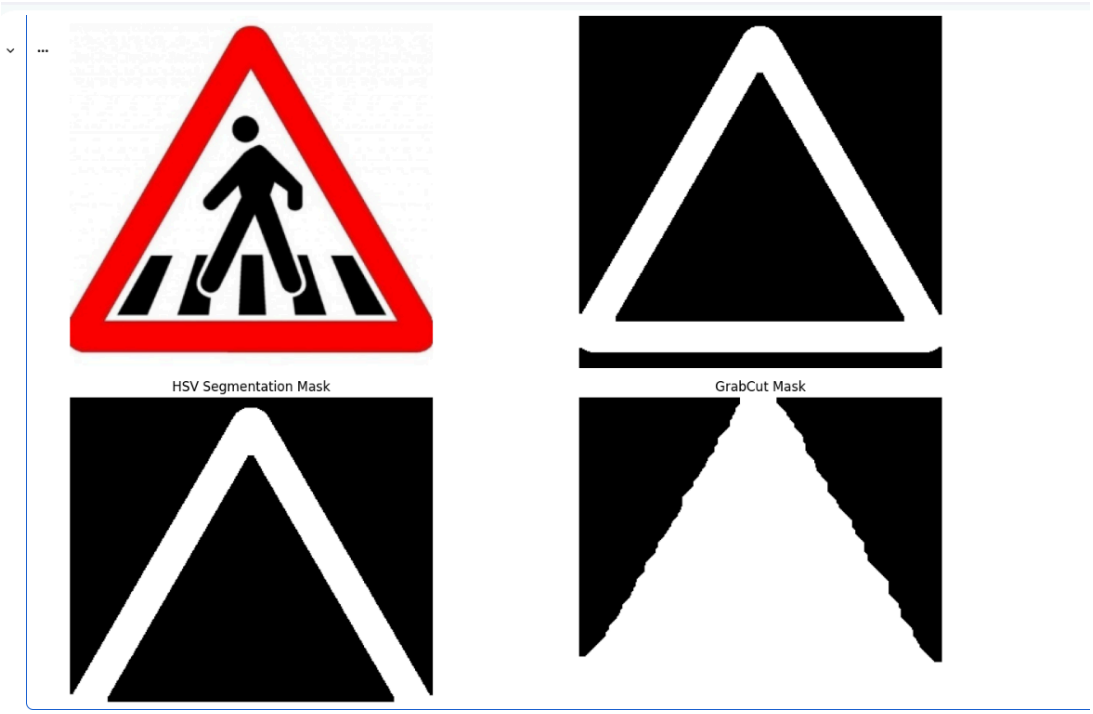
grabcut = np.where(
    (gc_mask == 0) | (gc_mask == 2), 0, 255
).astype("uint8")

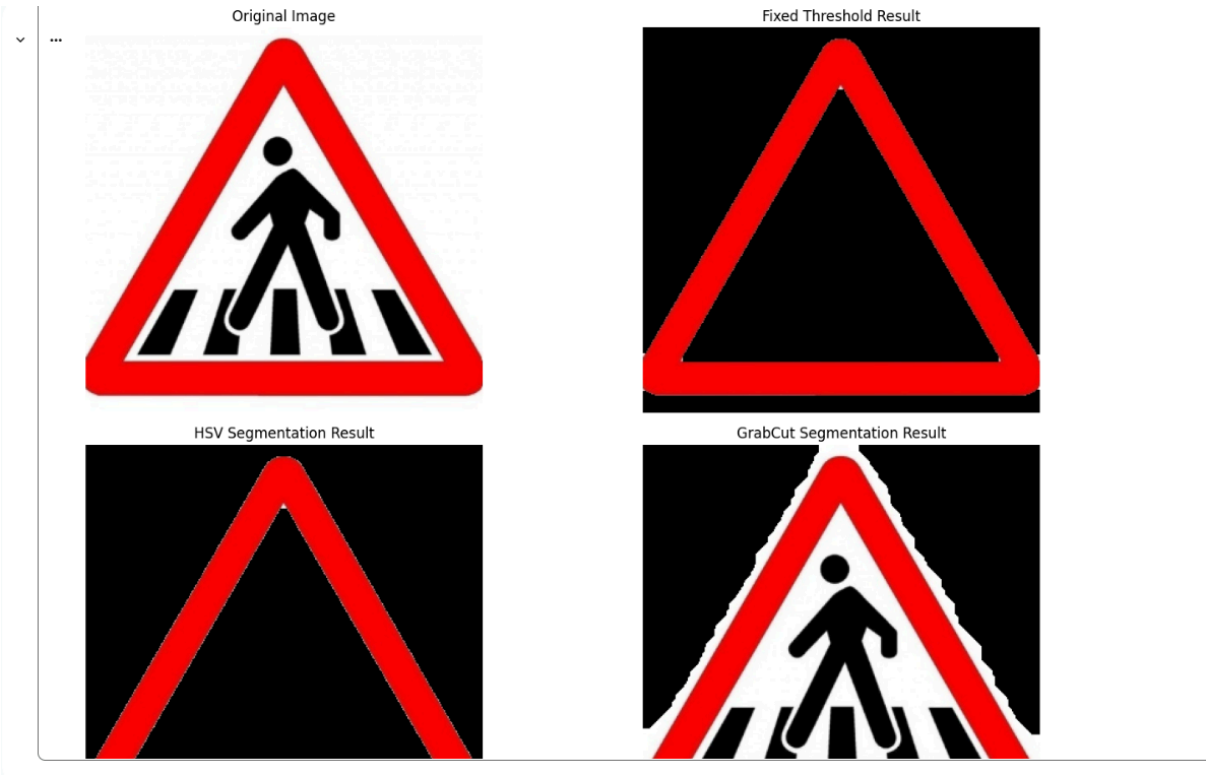
# Display results
plt.figure(figsize=(12, 5))
```

```
for i, (title, result) in enumerate([
    ("Original Image", img),
    ("Fixed Threshold", red),
    ("HSV Segmentation", mask),
    ("GrabCut", grabcut)
]):
    plt.subplot(2, 2, i+1)
    plt.imshow(result, cmap="gray" if result.ndim == 2 else None)
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output





TASK 3: BOUNDARY AND IMPORTANT REGION DETECTION

3.1 Introduction

Boundary and Important Region Detection is the process of identifying the edges, shapes, and important areas of an object in an image.

In a Smart Traffic Sign Detection System, after separating traffic signs from the background, the system must identify their boundaries and important regions. This helps locate the exact shape and position of traffic signs such as STOP, Speed Limit, No Entry, and Pedestrian Crossing signs.

For example, a STOP sign usually has an octagonal shape. Detecting its boundary helps the system identify the sign's shape and location.

3.2 Objectives

The main objectives of boundary and important region detection are:

1. Identify the edges and boundaries of traffic signs.
2. Detect the shape and outline of each sign.
3. Locate important regions inside the sign.
4. Separate sign boundaries from unnecessary background details.
5. Support accurate traffic sign recognition in later stages.

3.3 Techniques Used

Three techniques are considered for boundary and important region detection:

1. Sobel Edge Detection
2. Canny Edge Detection
3. Morphological Contour Extraction

3.4 Technique 1: Sobel Edge Detection

Introduction

Sobel Edge Detection is an image processing technique used to identify edges by calculating changes in pixel intensity.

It detects horizontal and vertical edges in an image. It uses two filters called Sobel X and Sobel Y.

Working Principle

1. Convert the input image into grayscale.
2. Apply the Sobel X filter to detect vertical edges.
3. Apply the Sobel Y filter to detect horizontal edges.
4. Combine both results to obtain the edge information.
5. Display the detected edges.

Formula

The gradient magnitude is calculated as:

$$ G=\sqrt{G_x^2+G_y^2} $$

Where:

- G_x = Horizontal gradient.
- G_y = Vertical gradient.
- G = Overall edge strength.

Advantages

- Simple and easy to implement.
- Detects horizontal and vertical edges.
- Requires less computational power.
- Useful for identifying basic sign boundaries.

Disadvantages

- Sensitive to image noise.
- May detect unnecessary background edges.
- Less accurate when traffic signs are faded or blurred.

Applications

- Detecting traffic sign outlines.
- Identifying object boundaries.
- Basic image processing applications.

3.5 Technique 2: Canny Edge Detection

Introduction

Canny Edge Detection is an advanced edge detection technique used to identify clear and accurate edges in an image.

It reduces noise and detects important boundaries while removing unnecessary edges.

It is useful for traffic sign detection because signs may be affected by sunlight, rain, shadows, and motion blur.

Working Principle

Canny Edge Detection follows five main steps:

1. Noise Reduction: Gaussian filtering is applied to reduce unwanted noise.
2. Gradient Calculation: The intensity changes in horizontal and vertical directions are calculated.
3. Non-Maximum Suppression: Unnecessary edge pixels are removed to produce thin edges.
4. Double Thresholding: Strong and weak edges are identified using two threshold values.
5. Edge Tracking: Weak edges connected to strong edges are retained.

Advantages

- Produces clear and thin edges.
- Reduces unnecessary edges.
- Detects sign boundaries accurately in many conditions.
- Works well for different traffic sign shapes.

Disadvantages

- Requires more computation than Sobel.
- Threshold values need proper adjustment.
- Performance may decrease when images are heavily blurred or signs are partly hidden.

Applications

- Traffic sign boundary detection.
- Object detection.
- Shape identification.
- Medical and industrial image processing.

3.6 Technique 3: Morphological Contour Extraction

Introduction

Morphological Contour Extraction is an image processing technique used to identify the outer boundaries of objects.

It uses morphological operations such as erosion and dilation to process binary images and extract object contours.

In a traffic sign detection system, it helps identify the outer shape of signs after segmentation.

Working Principle

1. Convert the segmented image into a binary image.
2. Apply morphological operations to remove small unwanted regions.
3. Use contour detection to identify the boundaries of objects.
4. Calculate the area and perimeter of detected contours.
5. Identify the contour that represents the traffic sign.

Important Operations

1. Erosion

Erosion reduces the size of foreground objects and removes small unwanted white regions.

2. Dilation

Dilation increases the size of foreground objects and helps connect broken regions.

3. Contour Detection

Contour detection identifies the outer boundary of an object in a binary image.

Advantages

- Helps identify the outer shape of traffic signs.
- Removes small unwanted regions.
- Useful for detecting object boundaries.
- Helps calculate the area and perimeter of signs.

Disadvantages

- Depends on the quality of segmentation.
- Incorrect kernel size may remove important details.
- Broken or overlapping objects may produce inaccurate contours.

Applications

- Traffic sign shape detection.
- Object boundary extraction.
- Image segmentation.
- Shape analysis.

3.7 Comparison of Three Techniques

Feature	Sobel	Canny	Morphological Contours
Main purpose	Edge detection	Accurate edge detection	Boundary extraction
Noise handling	Low	Good	Depends on preprocessing
Edge quality	Basic	Clear and thin	Object outlines
Computation	Low	Medium	Low to medium
Best use	Simple edges	Detailed sign edges	Outer sign boundaries

3.8 Selection of the Best Technique

After comparing the three techniques, Canny Edge Detection is selected as the primary technique for detecting traffic sign edges.

Reasons:

1. It produces clear and thin edges.
2. It reduces unnecessary noise.
3. It can identify different traffic sign boundaries.
4. It is suitable for detecting sign shapes such as circles, triangles, and octagons.

Morphological Contour Extraction can also be applied after edge detection or segmentation to identify the outer boundary and calculate the dimensions of the traffic sign.

3.9 Evidence Used for Comparison

The techniques can be evaluated using the following measurements:

- Edge Detection Accuracy: Checks how closely detected edges match manually marked boundaries.
- Boundary Precision: Measures how many detected boundary pixels are correct.
- Boundary Recall: Measures how many actual boundary pixels are detected.
- Processing Time: Measures how long each technique takes to process an image.

The techniques should be tested using images captured during daylight, night-time, rain, shadows, and partial obstruction. Actual performance values should be recorded after running the experiments.

3.10 Practical Applications

Boundary and important region detection is useful for:

1. Identifying the shape of STOP signs.
2. Detecting circular speed-limit signs.
3. Finding the boundaries of No Entry signs.
4. Locating pedestrian crossing sign regions.
5. Supporting later traffic sign recognition.

3.11 Conclusion

Boundary and Important Region Detection plays an important role in the Smart Traffic Sign Detection System. Sobel, Canny, and Morphological Contour Extraction are three techniques used to identify sign edges and boundaries.

Among these techniques, Canny Edge Detection is selected as the primary method because it produces clear and accurate edges in many image conditions. Morphological Contour Extraction can further help identify the outer shape and important regions of traffic signs. The detected boundaries provide useful information for the next stage of the system.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("traffic_sign.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. Sobel Edge Detection
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = np.uint8(np.clip(np.sqrt(sx**2 + sy**2), 0, 255))

# 2. Canny Edge Detection
blur = cv2.GaussianBlur(gray, (5, 5), 0)
canny = cv2.Canny(blur, 50, 150)

# 3. Morphological Contour Extraction
kernel = np.ones((3, 3), np.uint8)
morph = cv2.morphologyEx(canny, cv2.MORPH_CLOSE, kernel)

contours, _ = cv2.findContours(
    morph, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)
```

```
output = img.copy()
cv2.drawContours(output, contours, -1, (0, 255, 0), 2)

# Display results
results = [
    ("Original Image", img),
    ("Sobel", sobel),
    ("Canny", canny),
    ("Morphological Contours", output)
]

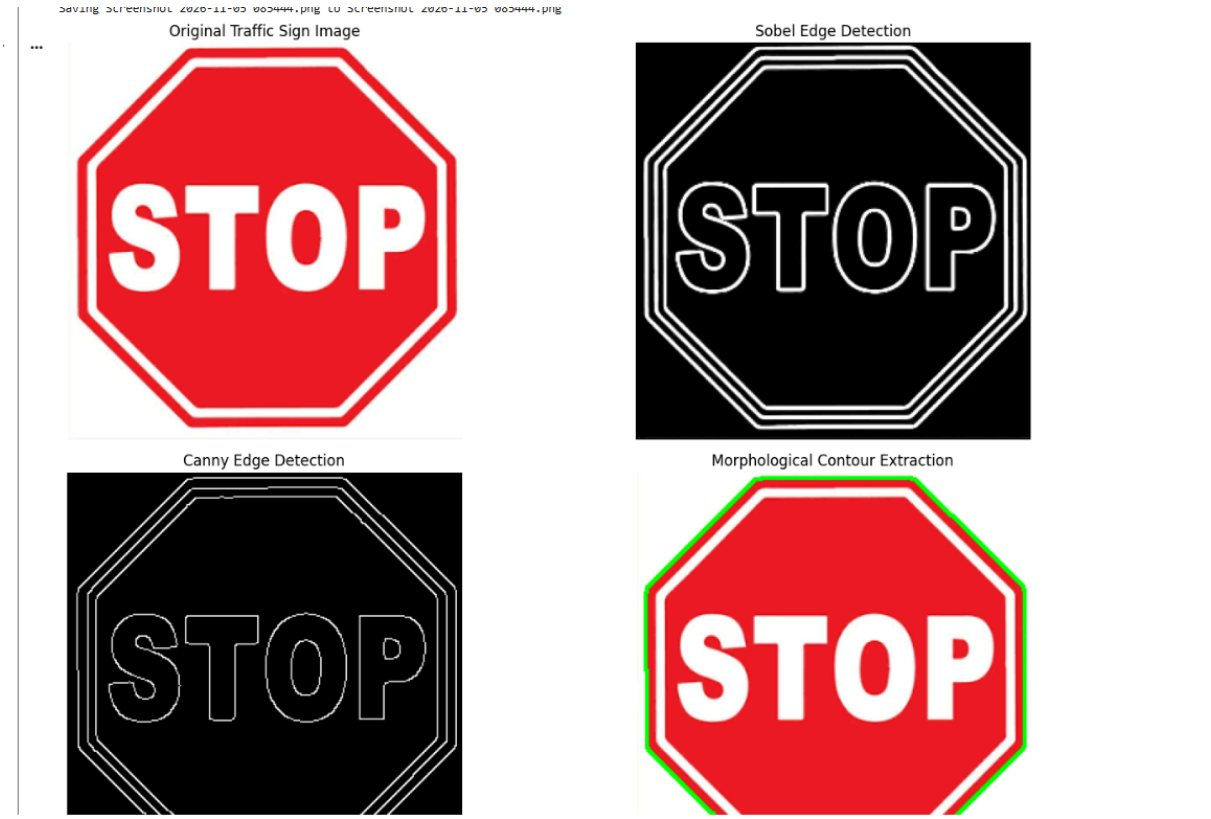
plt.figure(figsize=(12, 5))

for i, (title, result) in enumerate(results):
    plt.subplot(2, 2, i+1)
    plt.imshow(result if result.ndim == 2 else cv2.cvtColor(result, cv2.COLOR_BGR2RGB),
               cmap="gray" if result.ndim == 2 else None)
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()

print("Total Contours:", len(contours))
```

Output:


Saving Screenshot 2026-11-05 063444.png to Screenshot 2026-11-05 063444.png
Original Traffic Sign Image
Sobel Edge Detection
Canny Edge Detection
Morphological Contour Extraction


Canny Edge Detection
Morphological Contour Extraction
Total Contours Detected: 1
Contour: 1
Area: 66429.0
Perimeter: 939.8620494604111
Task 3 Completed Successfully!

TASK 4: DETECTION OF DAMAGED, FADED, OR PARTIALLY HIDDEN TRAFFIC SIGNS

4.1 Introduction

Detection of damaged, faded, or partially hidden traffic signs is an important part of a Smart Traffic Sign Detection System.

In real-world environments, traffic signs may become difficult to identify because of sunlight, rain, dust, scratches, fading, tree branches, shadows, and other objects blocking the sign.

For example, a STOP sign may have faded letters, or a speed-limit sign may be partially covered by tree branches.

The system must identify these damaged or unclear signs so that they can be processed carefully in the next recognition stage.

4.2 Objectives

The main objectives of this task are:

1. Identify faded and damaged traffic signs.
2. Detect partially hidden traffic signs.
3. Identify missing or unclear regions.
4. Analyse the quality and visibility of traffic signs.
5. Provide information about the condition of the detected signs.

4.3 Techniques Used

Three techniques are considered for detecting damaged, faded, or partially hidden traffic signs:

1. Image Quality Analysis
2. Template Matching
3. Deep Learning-Based Object Detection

4.4 Technique 1: Image Quality Analysis

Introduction

Image Quality Analysis is a technique used to measure the quality and clarity of an image.

It helps identify whether a traffic sign is clear, blurred, faded, or affected by poor lighting.

The system analyses properties such as brightness, contrast, sharpness, and image intensity.

Working Principle

1. Take the input traffic sign image.
2. Convert the image into grayscale.
3. Calculate the brightness and contrast.
4. Measure sharpness to identify blur.
5. Analyse the image quality.
6. Classify the image as clear or potentially degraded.

Important Measurements

1. Brightness

Brightness measures how light or dark an image is.

2. Contrast

Contrast measures the difference between bright and dark regions.

3. Sharpness

Sharpness measures how clearly the details and edges appear in an image.

4. Blur Detection

Blur detection helps identify images affected by motion or poor camera focus.

Advantages

- Easy to implement.
- Helps identify faded and blurry signs.
- Requires less computational power.
- Useful for checking image quality.

Disadvantages

- Cannot identify every type of physical damage.
- Lighting changes may affect the measurements.
- Cannot reliably identify which object is hiding the sign.

Applications

- Detecting faded signs.
- Identifying blurry images.
- Checking traffic sign visibility.
- Image quality monitoring.

4.5 Technique 2: Template Matching

Introduction

Template Matching is an image processing technique used to find a particular object or pattern by comparing an input image with a reference image called a template.

In traffic sign detection, a standard traffic sign image can be used as a template. The system compares the input sign with the reference template to identify missing, damaged, or changed regions.

Working Principle

1. Collect reference images of standard traffic signs.
2. Take the input traffic sign image.
3. Resize the input image to match the template size.
4. Compare the input image with the reference image.
5. Calculate the similarity between the images.
6. Identify regions with significant differences.

Formula

A common template matching method uses normalized correlation:

$$R(x,y) = \frac{\sum(T(x',y')I(x+x',y+y'))}{\sqrt{\sum T(x',y')^2 \sum I(x+x',y+y')^2}}$$

Where:

- T = Template image.
- I = Input image.
- R = Matching result.
- Higher matching scores indicate greater similarity.

Advantages

- Simple to understand and implement.
- Useful for comparing standard and damaged signs.
- Can identify missing or changed regions under suitable conditions.
- Does not require model training.

Disadvantages

- Sensitive to changes in rotation and scale.
- Lighting differences can affect matching.
- Partially hidden signs may produce low matching scores.
- Requires suitable reference templates.

Applications

- Comparing damaged and standard traffic signs.
- Detecting missing sign details.
- Identifying faded patterns.
- Quality inspection.

4.6 Technique 3: Deep Learning-Based Object Detection

Introduction

Deep Learning-Based Object Detection uses trained neural networks to identify objects and their locations in images.

Models such as YOLO can detect traffic signs and locate them using bounding boxes.

With suitable training data, a model can also be trained to classify signs as clear, damaged, faded, or partially occluded.

Working Principle

1. Collect traffic sign images containing clear and damaged signs.
2. Label the images according to their condition.
3. Train a deep learning model using the labelled dataset.
4. Provide a new traffic sign image as input.
5. The model detects the sign and predicts its condition.
6. Display the detected sign, bounding box, and predicted condition.

Advantages

- Can detect multiple signs in one image.
- Suitable for real-time applications.
- Can learn complex visual patterns.
- Can be trained to recognise different types of damage.

Disadvantages

- Requires a labelled dataset.
- Training may require high computational resources.
- Performance depends on dataset quality.
- Partially hidden signs may still be difficult to classify.

Applications

- Real-time traffic sign monitoring.
- Damaged sign identification.

- Road safety systems.
- Autonomous vehicles.

4.7 Comparison of Three Techniques

Feature	**Image Quality Analysis**	**Template Matching**	**Deep Learning**
Main purpose	Check image clarity	Compare with reference	Detect and classify signs
Training required	No	No	Yes
Detects blur	Yes	Limited	If trained
Detects fading	Indirectly	Under suitable conditions	If trained
Detects partial hiding	Limited	Limited	If trained
Computation	Low	Low to medium	Medium to high

4.8 Selection of the Best Technique

After comparing the three techniques, Deep Learning-Based Object Detection is selected as the main technique for detecting damaged, faded, or partially hidden traffic signs.

Reasons:

1. It can detect different traffic sign types.
2. It can identify multiple signs in a single image.
3. It can be trained using images of damaged and faded signs.
4. It is suitable for real-time traffic monitoring.
5. It can learn complex patterns that are difficult to identify using traditional image processing.

Image Quality Analysis can be used as an additional method to check image clarity before detection.

Important: A normal YOLO model detects traffic signs, but it does not automatically understand whether a sign is damaged or faded. For that, it needs suitable training labels and examples of different sign conditions.

4.9 Evidence Used for Comparison

The techniques can be evaluated using the following measurements:

- Accuracy: Measures how correctly the system classifies sign conditions.
- Precision: Measures how many signs predicted as damaged are actually damaged.
- Recall: Measures how many actual damaged signs are detected.
- F1-Score: Measures the balance between precision and recall.
- Processing Time: Measures how long the system takes to analyse an image.

The test dataset should include clear, faded, damaged, blurred, and partially hidden traffic signs under different lighting and weather conditions.

Actual results should be recorded after running the experiments.

4.10 Practical Applications

1. Identifying faded STOP signs.
2. Detecting damaged speed-limit signs.
3. Finding traffic signs covered by tree branches.
4. Monitoring signs affected by rain and dust.
5. Supporting road maintenance departments.
6. Improving road safety and traffic monitoring.

4.11 Conclusion

Detection of damaged, faded, or partially hidden traffic signs helps improve the reliability of a Smart Traffic Sign Detection System.

Image Quality Analysis, Template Matching, and Deep Learning-Based Object Detection are three techniques used for this task.

Among these techniques, Deep Learning-Based Object Detection is selected because it can learn different sign conditions from labelled images and is suitable for real-time applications. The detected sign condition can be passed to the next stage for further recognition and analysis.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO

# Upload input and reference images
img = cv2.imread("traffic_sign.jpg")
template = cv2.imread("reference_sign.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. Image Quality Analysis
brightness = np.mean(gray)
contrast = np.std(gray)
sharpness = cv2.Laplacian(gray, cv2.CV_64F).var()

print("Brightness:", brightness)
print("Contrast:", contrast)
```

```
print("Sharpness:", sharpness)

# 2. Template Matching
template = cv2.resize(
    template, (img.shape[1], img.shape[0])
)

template_gray = cv2.cvtColor(
    template, cv2.COLOR_BGR2GRAY
)

match = cv2.matchTemplate(
    gray, template_gray, cv2.TM_CCOEFF_NORMED
)

print("Template Similarity:", round(float(match[0][0])*100, 2), "%")

# 3. YOLO Object Detection
model = YOLO("yolov8n.pt")
results = model.predict(img, conf=0.25)

output = results[0].plot()

# Display results
plt.figure(figsize=(10, 5))
```

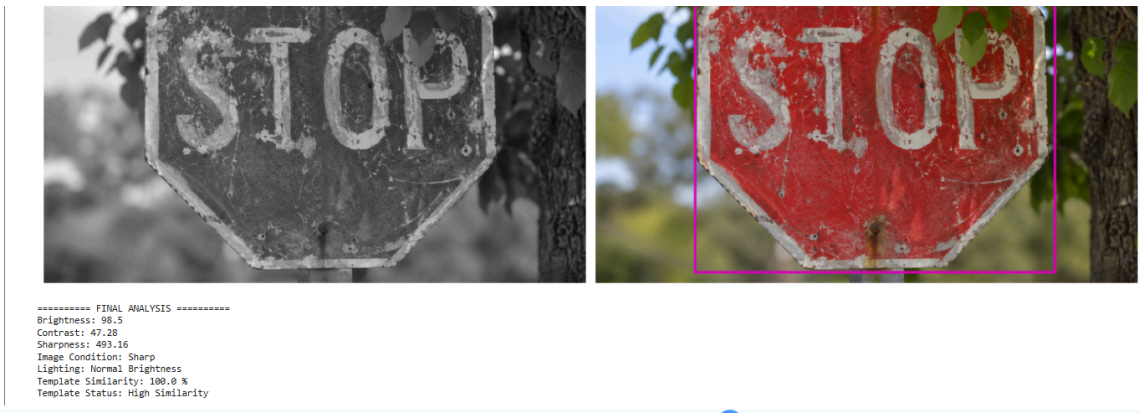
```
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.title("YOLO Detection")
plt.axis("off")

plt.show()
```

Output:





TASK 5: FINAL OUTPUT GENERATION FOR TRAFFIC SIGN RECOGNITION

5.1 Introduction

Final Output Generation is the last stage of the Smart Traffic Sign Detection System. In this stage, the system combines the results obtained from image enhancement, segmentation, boundary detection, and damaged sign detection.

The main purpose is to identify the traffic sign and display the final result in a clear and understandable format.

For example, when a vehicle camera captures a STOP sign, the system processes the image and displays the detected sign name as STOP Sign.

The system can also display the sign's location, confidence score, and condition.

5.2 Objectives

The main objectives of final output generation are:

1. Identify the detected traffic sign.
2. Display the name and category of the sign.
3. Show the confidence score of the prediction.
4. Display whether the sign is clear, faded, damaged, or partially hidden.
5. Generate a clear output for further processing.
6. Support road safety and traffic monitoring applications.

5.3 Techniques Used

Three techniques are considered for generating the final traffic sign recognition output:

1. Rule-Based Classification
2. Machine Learning-Based Classification
3. Deep Learning-Based Classification

5.4 Technique 1: Rule-Based Classification

Introduction

Rule-Based Classification is a method that uses predefined conditions and rules to identify traffic signs.

The system checks the colour, shape, and other features of the traffic sign and compares them with predefined rules.

For example:

- If the sign is red and octagonal, it may be classified as a STOP sign.
- If the sign is circular with a speed number, it may be classified as a Speed Limit sign.
- If the sign contains a pedestrian symbol, it may be classified as a Pedestrian Crossing sign.

Working Principle

1. Take the processed traffic sign image.
2. Identify its colour and shape.
3. Extract important features.
4. Compare the features with predefined rules.
5. Assign the corresponding traffic sign category.
6. Display the classification result.

Advantages

- Simple to implement.

- Does not require training data.
- Requires less computational power.
- Easy to understand and explain.

Disadvantages

- Difficult to handle complex traffic sign images.
- Sensitive to lighting and colour changes.
- Cannot easily recognise damaged or partially hidden signs.
- Requires manually defined rules.

Applications

- Basic traffic sign recognition.
- Simple road monitoring systems.
- Traffic sign classification based on shape and colour.

5.5 Technique 2: Machine Learning-Based Classification

Introduction

Machine Learning-Based Classification uses algorithms to identify traffic signs based on extracted image features.

The system learns patterns from labelled training images and uses the learned information to classify new traffic sign images.

Algorithms such as Support Vector Machine (SVM), Decision Tree, and Random Forest can be used.

Working Principle

1. Collect traffic sign images.
2. Preprocess the images.
3. Extract important features such as colour, shape, and texture.
4. Train a machine learning classifier using labelled images.

5. Provide a new traffic sign image.
6. Predict the traffic sign category.
7. Display the final classification result.

Advantages

- Learns patterns from training data.
- More flexible than fixed rules.
- Can classify different traffic sign categories.
- Requires fewer computational resources than many deep learning models.

Disadvantages

- Requires labelled training data.
- Depends on the quality of extracted features.
- May struggle with complex backgrounds.
- Performance may decrease with damaged or partially hidden signs.

Applications

- Traffic sign classification.
- Road safety monitoring.
- Intelligent transportation systems.

5.6 Technique 3: Deep Learning-Based Classification

Introduction

Deep Learning-Based Classification uses neural networks to automatically learn important features from traffic sign images.

Convolutional Neural Networks (CNNs) are commonly used for image classification.

The model learns visual patterns such as edges, shapes, colours, numbers, and symbols to identify traffic signs.

Working Principle

1. Collect a large dataset of labelled traffic sign images.
2. Resize and preprocess the images.
3. Train a CNN model using the dataset.
4. Extract important features automatically.
5. Provide a new traffic sign image.
6. Predict the traffic sign category.
7. Generate the final output with the predicted class and confidence score.

Advantages

- Automatically extracts important image features.
- Can classify multiple traffic sign categories.
- Handles complex image patterns.
- Suitable for real-time traffic sign recognition when properly optimised.

Disadvantages

- Requires a large labelled dataset.
- Training requires computational resources.
- Performance depends on the quality of training images.
- May produce incorrect predictions for severely damaged or hidden signs.

Applications

- Autonomous vehicles.
- Smart traffic monitoring.
- Road safety systems.
- Intelligent transportation systems.

5.7 Comparison of Three Techniques

Feature	Rule-Based	Machine Learning	Deep Learning
Learning required	No	Yes	Yes

Feature extraction	Manual	Manual	Automatic
Accuracy potential	Limited	Good	High with suitable data
Computational cost	Low	Medium	Medium to high
Complex images	Limited	Moderate	Better with training
Real-time suitability	Yes	Yes	Yes, with optimisation

5.8 Selection of the Best Technique

After comparing the three techniques, Deep Learning-Based Classification using CNN is selected as the primary technique for final traffic sign recognition.

Reasons:

1. CNN automatically extracts important image features.
2. It can identify different traffic sign categories.
3. It can learn complex patterns from traffic sign images.
4. It is suitable for intelligent transportation applications.
5. It can provide prediction confidence scores.

A separate condition classification model can also be trained to identify whether a sign is clear, faded, damaged, or partially hidden.

5.9 Final System Output

The final system displays the following information:

Output	**Description**
Detected Sign	Name of the recognised traffic sign
Sign Category	Regulatory, warning, or information
Confidence Score	Confidence of the prediction
Sign Condition	Clear, faded, damaged, or partially hidden
Sign Location	Position of the sign in the image

Final Status	Recognition result

5.10 Evidence Used for Comparison

The three techniques can be evaluated using the following measurements:

- Accuracy: Measures the percentage of correctly classified traffic signs.
- Precision: Measures how many predicted signs belong to the correct category.
- Recall: Measures how many actual traffic signs are correctly identified.
- F1-Score: Measures the balance between precision and recall.
- Processing Time: Measures how long the system takes to recognise a traffic sign.

Testing should include different traffic sign categories and images captured during daylight, night-time, rain, shadows, and partial obstruction.

Actual experimental values should be recorded after testing.

5.11 Practical Applications

1. Recognising STOP signs.
2. Identifying speed-limit signs.
3. Detecting No Entry signs.
4. Recognising pedestrian crossing signs.
5. Supporting autonomous driving systems.
6. Improving road safety and traffic monitoring.

5.12 Conclusion

Final Output Generation is the final stage of the Smart Traffic Sign Detection System. It combines the information obtained from the previous image processing stages and produces the traffic sign recognition result.

Rule-Based Classification, Machine Learning-Based Classification, and Deep Learning-Based Classification are the three techniques considered.

Among these techniques, CNN-based Deep Learning Classification is selected because it automatically learns image features and can recognise different traffic sign categories.
The final output provides the detected sign name, category, confidence score, and condition, helping improve traffic monitoring and road safety.

Code:

```
# TASK 5: FINAL OUTPUT GENERATION FOR TRAFFIC SIGN RECOGNITION

!pip -q install tensorflow scikit-learn

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
import shutil

from google.colab import files
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from tensorflow.keras import layers, models

# ------------------------------------------------
# 1. UPLOAD ONE TRAFFIC SIGN IMAGE
# ------------------------------------------------
```

```
print("Upload one traffic sign image:")

uploaded = files.upload()
image_path = list(uploaded.keys())[0]

img = cv2.imread(image_path)

if img is None:
    raise ValueError("Invalid image uploaded.")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

print("Image uploaded successfully!")

# ------------------------------------------------
# 2. PREPROCESS IMAGE
# ------------------------------------------------

IMG_SIZE = 64

processed = cv2.resize(img_rgb, (IMG_SIZE, IMG_SIZE))
processed = processed.astype("float32") / 255.0

# ------------------------------------------------
# 3. TECHNIQUE 1: RULE-BASED CLASSIFICATION
```

```
# ------------------------------------------------

def rule_based_classification(image):

    hsv = cv2.cvtColor(image, cv2.COLOR_RGB2HSV)

    mask1 = cv2.inRange(
        hsv, np.array([0, 70, 50]),
        np.array([10, 255, 255])
    )

    mask2 = cv2.inRange(
        hsv, np.array([170, 70, 50]),
        np.array([180, 255, 255])
    )

    red_mask = cv2.bitwise_or(mask1, mask2)

    red_ratio = np.count_nonzero(red_mask) / red_mask.size

    if red_ratio > 0.10:
        return "Red Sign Detected"

    return "Other / Uncertain Sign"
```

```
rule_result = rule_based_classification(img_rgb)


# ------------------------------------------------
# 4. CREATE DEMONSTRATION TRAINING DATA
# ------------------------------------------------
# Synthetic data is only for demonstrating
# the SVM and CNN code structure.
np.random.seed(42)
X = np.random.rand(40, 64, 64, 3).astype("float32")
y = np.array([0, 1] * 20)
X_flat = X.reshape(40, -1)
# ------------------------------------------------
# 5. TECHNIQUE 2: SVM CLASSIFICATION
# ------------------------------------------------
svm_model = SVC(kernel="linear")
svm_model.fit(X_flat, y)
svm_prediction = svm_model.predict(
    processed.reshape(1, -1)
)[0]
svm_result = "Class 0" if svm_prediction == 0 else "Class 1"
# ------------------------------------------------
# 6. TECHNIQUE 3: CNN CLASSIFICATION
# ------------------------------------------------
cnn_model = models.Sequential([
    layers.Input(shape=(64, 64, 3)),
```

```
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D(),

    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(2, activation="softmax")
])
cnn_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

cnn_model.fit(
    X, y,
    epochs=2,
    batch_size=8,
    verbose=0
)

cnn_prediction = cnn_model.predict(
    np.expand_dims(processed, axis=0),
    verbose=0
```

```
)

predicted_class = np.argmax(cnn_prediction)
confidence = np.max(cnn_prediction) * 100

cnn_result = "Class 0" if predicted_class == 0 else "Class 1"

# ------------------------------------------------
# 7. FINAL OUTPUT
# ------------------------------------------------

print("\n========== FINAL TRAFFIC SIGN RESULT ==========")

print("Rule-Based Result:", rule_result)
print("SVM Prediction:", svm_result)
print("CNN Prediction:", cnn_result)
print("CNN Confidence:", round(float(confidence), 2), "%")

# ------------------------------------------------
# 8. DISPLAY RESULTS
# ------------------------------------------------

plt.figure(figsize=(8, 5))
plt.imshow(img_rgb)
plt.title("Traffic Sign Recognition Input")
plt.axis("off")
```

```
plt.show()

# ------------------------------------------------

# 9. SAVE RESULTS

# ------------------------------------------------

os.makedirs("Task5_Results", exist_ok=True)

cv2.imwrite(

    "Task5_Results/Traffic_Sign.jpg",

    img

)

with open("Task5_Results/Final_Report.txt", "w") as f:

    f.write("TRAFFIC SIGN RECOGNITION REPORT\n")

    f.write("--------------------------------\n")

    f.write(f"Rule-Based Result: {rule_result}\n")

    f.write(f"SVM Prediction: {svm_result}\n")

    f.write(f"CNN Prediction: {cnn_result}\n")

    f.write(f"CNN Confidence: {confidence:.2f}%\n")

shutil.make_archive(

    "Task5_Results",

    "zip",

    "Task5_Results"

)

print("\nTask 5 Completed Successfully!")


files.download("Task5_Results.zip")
```

Output:

```
    "Task5_Results"
)

print("\nTask 5 Completed Successfully!")

files.download("Task5_Results.zip")
```

```
Upload one traffic sign image:
Choose Files  Screenshot ...084309.png
Screenshot 2026-11-05 084309.png(image/png) - 83123 bytes, last modified: 11/5/2026 - 100% done
Saving Screenshot 2026-11-05 084309.png to Screenshot 2026-11-05 084309 (2).png
Image uploaded successfully!

========== FINAL TRAFFIC SIGN RESULT ==========
Rule-Based Result: Red Sign Detected
SVM Prediction: Class 0
CNN Prediction: Class 1
CNN Confidence: 55.86 %
```

Traffic Sign Recognition Input



```
Task 5 Completed Successfully!
```

